Ders 9 Genişletilmiş Notu: Yığın ve Kuyruk Veri Yapıları, Infix-Postfix Dönüşümü

Ders 9 Genişletilmiş Notu: Yığın ve Kuyruk Veri Yapıları, Infix-Postfix Dönüşümü

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Derse Giriş: İki Temel Veri Yapısı
- 2. Stack Kavramı: Tek Uçtan Erişim
- 3. Stack İşlemleri
- 4. Stack Overflow ve Stack Underflow
- 5. Stack'in Kullanım Alanları
- 6. Stack Nasıl Oluşturulur?
- 7. Dizi ile Stack: Yardımcı Fonksiyonlar, Push ve Pop
- 8. Örnek: String'i Ters Çevirme
- 9. Örnek: Palindrom Kontrolü
- 10. Linked List ile Stack
- 11. Örnek: Parantez Dengesi Kontrolü
- 12. Infix ve Postfix Gösterim
- 13. Infix'ten Postfix'e: Parantezleme Yöntemi
- 14. Infix'ten Postfix'e: Stack Yöntemi
- 15. Postfix İfadeyi Hesaplama
- 16. Büyük Örnek ve Hocanın Hesap Hatası
- 17. Stack İşlemlerinin Karmaşıklığı
- 18. Kuyruk (Queue) Veri Yapısı
- 19. Dizi ile Kuyruk ve Dairesel Dizi
- 20. Linked List ile Kuyruk
- 21. Kuyruk İşlemlerinin Karmaşıklığı

1. Derse Giriş: İki Temel Veri Yapısı

Hoca derse dört haftanın geride kaldığını, beşinci haftaya başlandığını söyleyerek girdi. Bu derste iki temel veri yapısı tanıtıldı: stack ve queue (kuyruk).

Stack'in Türkçe karşılığı "yığın"dır. Hoca Türkçesini pek kullanmadıklarını, "stack" adını tercih ettiklerini söyledi. Sebebi, iki hafta sonra görülecek heap veri yapısının da Türkçeye bazen "yığın" ya da "yığıt" olarak çevrilmesidir. İki yapı Türkçede birbirinin yerine kullanılıp karışabiliyor. Bu yüzden derste "stack" adı kullanıldı.

Derste stack'in ne olduğu, ona özgü işlemler, bu işlemlerin özellikleri ve bilinen örnek uygulamaları görüldü. Ardından aynı şekilde queue ele alındı.

2. Stack Kavramı: Tek Uçtan Erişim

Hoca stack'i bir veri tutma organizasyonu olarak tanımladı. Tek özelliği, veri haznesine yalnızca tek bir uçtan erişilmesidir. Bir veri öbeğine sadece tek uçtan erişip kullanabiliyorsak ve

veri yapısı buna izin verecek şekilde tasarlandıysa, buna stack denir.

Hocanın benzetmesi masanın üzerine üst üste konmuş kitaplardı. En altta mavi bir kitap var. Üzerine turuncu, sonra mor, yeşil, kırmızı kitaplar konmuş. Alabileceğiniz ilk kitap en üstteki kitaptır. Aradan bir kitabı çekip alamazsınız, çünkü üstünde başka kitaplar vardır ve düzeni bozarsınız. En üstteki kitabı aldığınızda bir sonraki kitap en üste çıkar ve onu kullanabilirsiniz. Böylece masaya ilk koyduğunuz mavi kitap, en son erişebileceğiniz kitap olur.

Stack genellikle ağzı (tepesi) açık, altı kapalı bir hazne olarak çizilir. Bu yapı **LIFO** (last in, first out, son giren ilk çıkar) ya da **FILO** (first in, last out, ilk giren son çıkar) olarak adlandırılır. Elemanlar aynı tipte verilerdir. Yeni eleman yalnızca en üste eklenebilir. Kullanmak istediğimizde de yalnızca en üstteki elemana erişebiliriz, alttakilere erişemeyiz.

3. Stack İşlemleri

Böyle bir veri yapısında iki temel işleme ihtiyaç vardır: yeni eleman ekleme ve bir elemanı alıp kullanma.

- **push:** Stack'e bir eleman basmak (eklemek).
- **pop:** Stack'ten erişilebilen tek elemanı alıp kullanmak (çıkarmak).

Push ettikçe elemanlar tek tek üste eklenir. Pop ettikçe en son eklenen ilk alınır. Böylece veriler eklenme sırasının tersiyle kullanılır.

Hoca şunu özellikle vurguladı: Stack'e üçüncü sırada eklenen elemanı doğrudan alamayız. Beşinci sırada eklenen elemanın değerini de doğrudan okuyamayız. Elimizde yalnız push ve pop vardır.

> **Hocanın vurgusu:** Stack'te ortadaki bir elemana doğrudan erişilemez. Yalnız push ve pop ile ekleme-çıkarma yapılabilir.

Bazen en üstteki elemanın değerine bakmak, ama onu stack'ten çıkarmamak isteyebiliriz. Bunun için **peek** ya da **top** fonksiyonu tanımlanır. Hoca bunu "bakmak, göz atmak" diye Türkçeleştirdi.

Bu işlemlerin güvenli çalışması için bazı yardımcı (hizmet) fonksiyonlara da ihtiyaç vardır. Stack boşsa daha fazla eleman çıkaramayız. Stack ağzına kadar doluysa yeni eleman ekleyemeyiz. Bu kontroller için:

- **isEmpty:** Stack boş mu? Pop etmeden önce kontrol edilir.
- **isFull:** Stack dolu mu? Push etmeden önce kontrol edilir.

4. Stack Overflow ve Stack Underflow

Hoca tahtada boş bir stack çizdi. Kapasitesi 6 göz olsun. Sırayla 1, 2, 3, 4, 5 ve 6 push edildi. Her adımda yalnız en üstteki elemana erişilebiliyor. Örneğin 3 push edildiğinde stack'te 1, 2, 3 vardır, ama yalnız 3 kullanılabilir. 6 push edildiğinde stack tamamen dolar.

Altı eleman içerideyken 7. eleman push edilmeye çalışılırsa stack "dolar, taşar". Buna **stack overflow** denir.

- Statik boyutlu bir stack'te stack overflow yaşanabilir.
- Dinamik boyutlu bir stack teorik olarak hiç dolmaz. Ama fiziksel makinelerde bellek sınırlıdır. Belli bir noktadan sonra yeni eleman fiziksel olarak belleğe eklenemez.
- Bu yüzden derleyicilerde yazdığımız kodlar çalışırken bazen "stack overflow" hatası verip olağan dışı şekilde sonlanır ve kontrol işletim sistemine geri verilir.

Hoca, algoritmik sorunlarda sıkça başvurulan meşhur forum sitesinin adının da buradan geldiğini hatırlattı.

isFull kontrolü yapılırsa, yani stack doluyken eklemeye izin verilmezse, kontrollü bir yapı kurulmuş olur ve stack overflow yaşanmaz. Bu durumda eleman ekleyemediğimize göre pop ederek elemanları kullanabiliriz:

- İlk pop en üstteki 6'yı alır. Tepede 5 kalır, bir göz boşalır.
- Sonraki pop'lar sırasıyla 5, 4, 3, 2 ve 1'i alır.

- Sonunda stack'te hiç eleman kalmaz.

Boş stack'ten bir pop daha yapılmak istenirse içeride eleman yoktur. isEmpty'den true geldiğinde durmak ya da "stack boş" diye bir hata döndürmek gerekir. Bu kontrol yapılmazsa ve boş stack'ten sanki bellekteki bir gözü çeker gibi eleman alınmaya çalışılırsa **stack underflow** durumu yaşanır.

5. Stack'in Kullanım Alanları

Hoca stack'in çok kullanılan bir veri yapısı olduğunu, pek çok algoritmanın içinde işleri çok kolaylaştırdığını söyledi.

5.1 Fonksiyon Çağrı Yığını (Function Call Stack)

Program işletimi sırasında, özellikle fonksiyon dallanmalarında stack çok kullanılır. Kodda bir fonksiyon çağrısı yaptığınızı düşünün. Fonksiyona dallanmadan önce `main` içinde nerede kaldığınızı, nereden devam edeceğinizi ve değişkenlerinizin değerlerini bir yerde tutmanız gerekir. Bu yer **function call stack** yapısıdır. Dönülecek yer ve yerel değişkenlerin değerleri bir paket olarak buraya yazılır.

Çağrılan fonksiyon `f` özyinelemeli (recursive) olsun ve kendini başka bir parametreyle tekrar çağırsın. Bu yeni çağrıda da `f`'nin o anki durumu, yerel değişkenleri ve hangi satırdan devam edileceği tutulmalıdır. Bu da bir fonksiyon çağrısı olduğu için stack'e push edilir. Çağrılar böyle devam eder.

Özyinelemenin temel durumu işletildiğinde geri dönüş başlar. Nereye dönülecek? Bir önceki çağrının durumuna. Bu durum stack'in en üstündedir. Hoca çağrıları f_0, f_1, f_2, f_3 diye adlandırdı. Son çağrı bitince f_3'ün paketi pop edilir ve kaldığı satırdan devam edilir. Sonra f_2, f_1 ve f_0 sırasıyla pop edilir. Stack'te hiçbir fonksiyon dallanması kalmadığında `main` fonksiyonuna dönülmüş olur ve kod kaldığı yerden devam eder. Stack, nerede kalındığını ters sırayla kolayca takip etmeyi sağlar.

5.2 Diğer Kullanım Alanları

- **İfade ayrıştırma (parsing):** Derleyicilerde aritmetik ifadelerin ayrıştırılmasında stack kullanılır. Özellikle infix ve postfix gösterimlerinin birbirine dönüşümünde işleri çok kolaylaştırır.
- **Kod kontrolü:** Kodda bloklar için küme parantezi, fonksiyon parametreleri için normal parantez, dizi indisi için köşeli parantez vardır. Bunların doğru sayıda, doğru sırada ve iç içe yapıyı koruyarak açılıp kapanması gerekir. Derleyici bu kontrolü stack ile çok basit yapar.
- **Ters çevirme:** Bir dizinin elemanlarını tersine çevirmek stack ile çok rahattır.
- **Hanoi Kuleleri:** Özyinelemeli bir çözümü vardır. Karmaşıklığı 2^n olsa da stack ile çok basit bir şekilde gerçeklenebilir. Hoca bu problemi özyinelemeli fonksiyonlar dersinde göstermişti.
- **N vezir (N-queen) problemi:** Birbiriyle çatışmayan n veziri $n \times n$'lik satranç tahtasına yerleştirme problemidir. Kalınan yere geri dönerek ilerleyen geri izleme (backtracking) yaklaşımı stack ile çok kolaylaşır.

Hoca stack'i, algoritmik karmaşıklığı ve işlem yükünü azaltmak için sıkça kullanılan yardımcı bir araç olarak özetledi.

6. Stack Nasıl Oluşturulur?

Stack oluşturmanın iki yolu vardır:

- **Statik dizi ile:** Sabit boyutlu bir stack oluşur. Bir öğrenci "hocam bunu malloc ile dinamik yapabilirim" diyebilir. Hoca bunun yapılabileceğini, ama şu aşamada tartışmayacağını söyledi.
- **Linked list ile:** Dinamik, yani değişken boyutlu ve teorik olarak hiç dolmayan bir stack oluşur.

Linked list ile kurulan stack de pratikte sınırsız değildir. Bir mimaride adreslenebilen belleğin fiziksel bir sınırı vardır. Yeterince özyinelemeli çağrı yapıldıktan sonra stack için ayrılan bellek dolup taşabilir. Kod kontrolsüz işletiliyorsa stack overflow hatası alınır ve program anormal şekilde sonlanır.

6.1 Dizi ile Kurulum

Hoca diziyi bu kez dikine çizdi. `MAX` boyutu 8 olsun. İndisler 0'dan 7'ye, yani $n-1$'e kadardır.

Stack'te nerede olduğumuzu gösteren bir işaretçi değişkene ihtiyaç vardır. Buna **top pointer** denir. Başlangıçta stack boştur. top, 0. gözün de altını gösterecek şekilde -1 yapılır.

- top -1 ise stack boştur ve pop edilemez. isEmpty bu yüzden `return top == -1` kadar basittir.
- Push etmeden önce top'un $n-1$'e eşit olup olmadığına bakılır. Eşitse stack doludur.
- Push: önce top bir artırılır, sonra yeni eleman (örneğin 6, sonra 4) top'un gösterdiği göze yazılır.
- Pop: top'un gösterdiği elemanı al, sonra top'u bir azalt.

6.2 Linked List ile Kurulum

Yine bir pointer gerekir. Stack boşken pop edilemez. Boşken bir eleman push edilirse bir düğüm yaratılır ve top (ya da head) bu düğümü gösterir. Yeni bir eleman push edildiğinde yeni bir düğüm oluşturulur, bu düğümün pointer'ı bir öncekini gösterir ve top yeni düğüme güncellenir. Böylece liste yukarı doğru büyür.

Pop için top bir sonraki düğüme kaydırılır, eski düğümün değeri kullanılır ve eski düğüm `free` edilir. Hoca bunun geçen haftaki linked list'in aynısı olduğunu, yalnızca fazladan bir top pointer ile stack biçiminde düzenlendiğini söyledi.

7. Dizi ile Stack: Yardımcı Fonksiyonlar, Push ve Pop

7.1 isEmpty, isFull ve peek

Top pointer varsa bu fonksiyonları dizi ile yazmak çok kolaydır.

- **isEmpty:** top -1 ise stack boştur, `return 1` (true). Değilse top bir yeri gösteriyordur, eleman vardır, `return 0` (false).
- **isFull:** top en üst indise eşitse stack doludur, `return 1`. Değilse araya bir yerde ya da boştur, `return 0`.
- **peek:** `stack[top]` değeri döndürülür. Değer pop edilmez, yalnız bakılır.

```c
int isEmpty(void) {
    if (top == -1) return 1;
    return 0;
}

int isFull(void) {
    if (top == MAX - 1) return 1;
    return 0;
}

int peek(void) {
    return stack[top];
}
```

> **Ders sonrası not:** Hoca isFull'u anlatırken "top pointer maxSize'a eşitse doludur" dedi. Push akış diyagramında da kontrol "top max'a eşitse" diye geçti. İndisler $0..MAX-1$ olduğu için doğru kontrol `top == MAX - 1`'dir. Hoca dizi kurulumunu anlatırken de "$n-1$'e eşit mi diye bakarım" demişti.

Hoca peek'in çok fazla kullanılmasının önerilmediğini söyledi. Stack'teki asıl amaç ekleme-çıkarma organizasyonunu sağlamaktır. Bu yüzden push ve pop üzerinden gitmek daha doğ-

rudur. peek belki en üstteki elemanın beklenen eleman olup olmadığını kontrol etmek için kullanılabilir.

Soru: peek'e bakmadan pop ederek de o değere bakamaz mıyız?

Cevap: Bakılabilir, ama o zaman eleman stack'ten çıkarılmış olur. Artık o eleman stack'te kalmaz.

Soru: Aradığımız elemanı if-else ile kontrol edersek önce en üsttekine bakarız. O değilse altındakine bakmamız gerekir. O zaman zaten pop etmiş olmuyor muyuz?

Cevap: Alttaki elemana doğrudan bakılamaz, çünkü yalnız top bilinir. Stack'teki başka bir indis bilinmez. Pop etmek gerekiyorsa pop edilir. Hoca "önce peek et, sonra pop et" demediğini, yalnız gerekiyorsa peek diye bir işlem kurgulanabileceğini söyledi.

> **Hocanın vurgusu:** En önemli iki işlem push ve pop'tur. Push ve pop sırasında overflow ve underflow kontrol edilmelidir. Bunun için isEmpty ve isFull adlı iki hizmet fonksiyonu yazılır. "Olay bu." peek ise çok nadiren, belki kontrolde gerekir.

7.2 Push

Hoca push işlemini akış diyagramı üzerinden anlattı. Girdi olarak `MAX` boyutu, stack'teki elemanlar ve push edilecek eleman okunur. Başlangıçta top -1'dir. Sonra kontrol yapılır:

- top en üst gözü gösteriyorsa isFull true döndürür. "Stack full" denir ve yeni eleman eklenmez.
- Yer varsa top bir artırılır: `top = top + 1`. Sonra yeni eleman `stack[top] = data` ile yazılır.

Push fonksiyonu mutlaka bir veri almalıdır. Derste basitlik için tamsayı kullanıldı. Hoca elemanların struct da olabileceğini söyledi.

7.3 Pop

Pop'ta bunun tersi yapılır. Önce top -1 mi, yani stack boş mu diye bakılır. Boşsa "stack empty" denip bir hata verilir.

Boş değilse `stack[top]` değeri dışarıdaki geçici bir `data` değişkenine kopyalanır. Sonra top bir azaltılır. Fonksiyon bir değer döndüreceği için `int` tipindedir ve parametre almaz. Boşsa `-1` gibi bir hata kodu döndürülebilir.

```
void push(int data) {
    if (!isFull()) {
        top = top + 1;
        stack[top] = data;
    } else {
        printf("Stack full\n");
    }
}

int pop(void) {
    int data;
    if (!isEmpty()) {
        data = stack[top];
        top = top - 1;
        return data;
    }
    printf("Stack empty\n");
    return -1;   /* hata kodu */
}
```

Hoca önemli bir ayrıntıya dikkat çekti. Pop sonrasında değer dizide fiziksel olarak durmaya devam eder. Değer silinmedi, `stack[top] = 0` gibi bir şey de yapılmadı. Ama top artık bir önceki elemanı gösterir. Yeni bir pop yapılırsa o önceki eleman alınır. Yeni bir push yapılırsa top bir artırılır ve eski değerin üzerine yazılır.

Özetle dizi ile stack'te push etmeden önce isFull, pop etmeden önce isEmpty kontrol edilir. Sonra top pointer artırılıp ya da azaltılıp kullanılır.

8. Örnek: String'i Ters Çevirme

Elinizde "merhaba" gibi bir string olsun ve bunu tersine çevirmek isteyin. Bir öğrenci normal çözümü söyledi: $n - 1$'den 0'a bir döngüyle yazdırmak ya da yeni bir diziye taşımak. Hoca bunun doğru olduğunu, ama stack ile de çok basit yapılabileceğini söyledi.

"merhaba"nın sonunda null karakter (`'\0'`) olduğu bilinir.

1. Null karakter görülene kadar her karakter stack'e push edilir: m, e, r, h, a, b, a.
2. Null karakter görülünce string bitmiştir.
3. Stack boşalana kadar bütün elemanlar pop edilir ve çıktı dizisine yazılır (ya da ekrana basılır).

İlk pop `a`'yı verir. Sonra top `b`'yi gösterir ve `b` alınır. Ardından a, h, r, e, m gelir. top -1 olunca durulur. Sona null karakter eklenir. Sonuç "abahrem"dir.

Kod iki döngüden ibarettir: "null değilken karakteri push et", sonra "boş değilken karakteri pop et". Karmaşıklık yine $O(n)$'dir, ama çözüm daha basit ve anlaşılırdır.

9. Örnek: Palindrom Kontrolü

Bir string'in palindrom olup olmadığı kontrol edilmek isteniyor. Hoca öğrencilerden palindrom kelime istedi. "kabak" ve "kayak" söylendi, "kabak" seçildi.

Bir öğrenci çözümü önerdi: String'in yarısına kadar karakterler stack'e atılır, sonraki karakterler stack'ten çekilen karakterlerle karşılaştırılır. Hoca "yarısı nedir?" diye sordu. Öğrenci "b'ye kadar k ile a'yı atarız, b'yi atmayız" dedi. Çünkü palindrom olması için ortadaki karakterin öncesi ile sonrasının birbirinin tersi olması gerekir.

Başka bir öğrenci bunu algoritmik olarak ifade etti: "kabak" 5 karakterdir. $5/2$ tamsayı bölmesi 2 verir. 2 karakter push edilir, 3. karakter atlanır, sonra pop etmeye başlanır. Hoca bunun daha formal bir tanım olduğunu söyledi. Ortadaki karakter "don't care" karakteridir. Push edilse bile kendisiyle karşılaştırılacağı için boş bir kontrol olur. Bu yüzden atlanır.

kabak:

1. k push edilir, a push edilir.
2. b atlanır.
3. a gelir, bir pop yapılır, çıkan a'dır. a = a.
4. k gelir, bir pop yapılır, çıkan k'dır. k = k.
5. String sonu (null) gelmiştir ve stack de boştur. Palindromdur.

tabak:

1. t ve a push edilir, b atlanır.
2. a gelir, pop edilen a'dır. Eşit.
3. k gelir, pop edilen t'dir. k ≠ t.

Eşitsizlik görüldüğü anda döngü kesilebilir. Sona kadar gitmeye gerek yoktur. "Not palindrome" çıktısı üretilir.

küllük: 6 karakterdir. $6/2 = 3$ olduğu için ilk üç karakter k, ü, l push edilir. Sonra 4., 5. ve 6. karakterler sırasıyla pop edilen karakterlerle karşılaştırılır: l = l, ü = ü, k = k. Sonda null görülüp stack'in boş olduğu da kontrol edilir. Palindromdur.

> **Ders sonrası not:** Hoca "kabak" örneğinin sonunda "is full'den de true geldi" dedi. Burada kastedilen kontrol isEmpty'dir. String bittiğinde stack'in boş olması, yani isEmpty'nin true dönmesi gerekir.

10. Linked List ile Stack

10.1 Yapı

Hoca 12, 8, 21, 33, 18 ve 40 değerlerinin push edildiği bir stack çizdi. top pointer en üstteki 40'ı gösteriyor. Sonra şekli sadeleştirdi: İlk push edilen eleman 33 olsun. 33, linked list'te son eleman olarak yer alır. Bir zamanlar top (head) orasıydı.

- 18 push edildi: Yeni düğüm yaratıldı. Düğümün pointer'ı eski top'a (33'e) bağlandı. top yeni düğüme güncellendi.
- 40 push edildi: Yine düğüm yaratıldı, 40 yazıldı. Pointer eski top'u (18'i) gösterdi. top yeni düğüme güncellendi.

Bu yapı için bir struct gerekir. İçinde veri ve kendinden önce push edilmiş düğümün adresini tutan `struct node *` tipinde bir `ptr` işaretçisi vardır. Yer değiştirmeler için top'a ek olarak `top1` ve `temp` adlı iki `struct node *` işaretçisi daha kullanılır.

İşlemler:

- **push ve pop:** Temel iki işlem.
- **create:** `top = NULL`. NULL gösteren bir stack yaratılmış olur.
- **destroy:** Stack'i yok etmek için.
- **stack count:** Eleman sayısını tutan bir sayaç. Hoca buna aslında hiç gerek olmadığını, kolaylık olsun diye konduğunu söyledi. Global bir değişken olarak ya da ayrı bir stack struct'ı içinde tutulabilir.
- **peek:** `return top->data`.
- **isEmpty:** top NULL ise stack boştur, `return 1`. Değilse `return 0`.

Linked list stack'inde isFull yoktur, çünkü teorik olarak dolmaz. Ama bir sayaç tutulup bir `MAX` sınırı konmak istenirse isFull kontrolü de yazılmalıdır.

10.2 Push

Hoca aradan sonra push ile devam etti. İki durum vardır:

- **Stack boşsa** (top NULL ise): `malloc` ile `struct node` tipinde yer ayrılır ve top bu düğümü gösterir. Düğümün pointer'ı NULL yapılır, veri yazılır.
- **Stack boş değilse:** `temp` adında geçici bir düğüm yaratılır. Pointer'ı bir önceki top'a bağlanır. Veri yazılır. Sonra `top = temp` ile top güncellenir.

Stack'e her zaman top pointer ile erişildiği unutulmamalıdır. top her zaman son eklenen (last in) elemanı gösterir. İstenirse sayaç da bir artırılır.

```
void push(int data) {
    if (top == NULL) {
        top = (struct node *)malloc(1 * sizeof(struct node));
        top->ptr = NULL;
        top->data = data;
    } else {
        temp = (struct node *)malloc(1 * sizeof(struct node));
        temp->ptr = top;
        temp->data = data;
        top = temp;
    }
    count++;
}
```

10.3 Pop

Pop için bir dönüş değişkeni (`ret`) ve top değişeceği için `top1` adında bir işaretçi daha gerekir. Hoca üç elemanlı bir stack çizdi. En alttaki düğüm NULL'u gösteriyor.

1. `top1` de top'u gösterir. top1 NULL ise stack'te eleman yoktur, `-1` hata kodu döndürülür.
2. `top1` bir ilerletilir, yani top'un altındaki düğümü gösterir.
3. Dönüş değeri olarak top'un gösterdiği düğümün verisi `ret`'e alınır. İstenirse yazdırılır.

4. top'un gösterdiği düğüm `free` edilir.
5. `top = top1` ile top bir alttaki düğüme güncellenir.

Hoca `top1`'in neden gerektiğini vurguladı: top `free` edilirken bir sonraki elemanın kaybedilmemesi için. Sonrasında `top1` de kaldırılabilir.

```
int pop(void) {
    int ret;
    top1 = top;
    if (top1 == NULL)
        return -1;          /* stack boş: hata kodu */
    top1 = top1->ptr;
    ret = top->data;
    free(top);
    top = top1;
    count--;
    return ret;
}
```

10.4 Destroy

Destroy işleminde `top1` NULL olana kadar düğümlerin hepsi `free` edilir. Böylece stack'te eleman varsa bile hepsi yok edilir.

11. Örnek: Parantez Dengesi Kontrolü

Stack'in kullanım alanlarında sayılan kod kontrolü örneği ele alındı. Verilen bir kod ifadesinde parantezler doğru mu, sayıları dengeli mi, iç içe doğru açılıp kapanmışlar mı? Bunu kontrol eden bir algoritma stack ile tasarlanır.

Hoca tahtaya bir dizinin en büyük elemanını bulan, köşeli parantez içinde `max(a)` çağrısı içeren bir atama ve arkasından küme parantezli bir kod bloğu yazdı. Kabaca `[ ( ) ]` ve sonra `{ }` biçimindeydi.

Algoritma girdiyi soldan sağa işler:

- Açma parantezi (`(`, `[`, `{`) görülünce stack'e push edilir.
- Kapama parantezi görülünce stack'ten bir pop yapılır. Gelen kapama ile çekilen açmanın tipi eşleşiyor mu diye bakılır. Eşleşmezse hata verilir.
- Bütün girdi işlendikten sonra stack boş mu diye bakılır. Boş değilse parantezler dengesizdir.

Örnek 1: `[ ( ) ]` ve `{ }`

1. `[` push edilir. Sonra `max`'tan sonraki `(` push edilir. top `(`'yi gösterir.
2. `)` gelir. Pop edilen `(`'dir. Tipler eşleşir.
3. `]` gelir. Pop edilen `[`'dir. Eşleşir. Stack boşalır.
4. `{` push edilir. Bloktaki işlemlerden sonra `}` gelir. Pop edilen `{`'dir. Eşleşir. top yine -1'dir.
5. Girdi biter, stack boştur. Parantezler doğru ve dengelidir.

Örnek 2: `[ ( ( ) ]`

1. `[`, `(` ve `(` push edilir.
2. `)` gelir, pop edilen `(`'dir. Eşleşir.
3. `]` gelir, pop edilen `(`'dir. Eşleşmez.

Daha fazla devam etmeye gerek yoktur. "Eşleşmiyor" sonucu döndürülür.

Örnek 3: `[ ( ( ) )`

1. `[`, `(` ve `(` push edilir.
2. `)` gelir, pop edilen `(`'dir. Uygun.
3. `)` gelir, pop edilen `(`'dir. Uygun.
4. Girdinin sonuna gelinmiştir.

Hoca burada durmanın yeterli olmadığını söyledi. Stack boş mu diye mutlaka kontrol edilmelidir. Stack'te hâlâ `[` vardır. Bu yüzden parantezler doğru sayıda açılıp kapanmamıştır, ifade dengesizdir.

Hoca bu algoritmanın sözde kodunun slaytta olduğunu, uzun uzun anlattığı için üzerinde durmayacağını söyledi.

12. Infix ve Postfix Gösterim

Stack'in çok kullanıldığı bir başka alan infix-postfix dönüşümüdür.

Infix gösterim insanların okuması için mantıklı olan gösterimdir. Operatörler operandların arasında (in) yer alır. İlkokuldan beri `3 + 2` biçiminde yazarız. Ama birden fazla işlem ve parantez olunca okunurluk karışır.

Hocanın örneği $3 + 2 \cdot 5 + 1$ idi. İnsan, çarpmanın toplamadan öncelikli olduğunu bildiği için önce $2 \cdot 5 = 10$'u hesaplar: $3 + 10 + 1 = 14$.

Derleyicinin insan gibi gözleri yoktur. İfadeyi ancak bir yönden başlayarak işleyebilir. Hoca bunu kutucuklarla anlattı:

- `3 + 2` için sorun yoktur. 3 birinci kutuya yazılır. `+` görülünce bir toplama yapılacağı anlaşılır. 2 gelince $3 + 2 = 5$ hesaplanır.
- $3 + 2 \cdot 5 + 1$ için: 3 ve `+` yazılır. 2 gelince $3 + 2 = 5$ yapılır. `*` gelir, 5 gelir, $5 \cdot 5 = 25$ olur. `+` ve 1 gelince $25 + 1 = 26$ bulunur.

Derleyici böyle çalışırsa "patlar", çünkü aritmetik işlemi yanlış yapmış olur. Doğru sonuç 14'tür.

Peki tek yönden çalışan derleyiciye ifadeyi nasıl okutmalıyız? **Postfix** gösterimde operatör, operandlarından sonra gelir. `3 2 +` "3 geldi, 2 geldi, bu iki operanda bu operatörü uygula" demektir.

$3 + 2 \cdot 5 + 1$ postfix'te `3 2 5 * + 1 +` olarak yazılır:

1. 3, 2 ve 5 gelir. Sonra bir operatör (*) görülür. Operatör kendinden önceki iki operandı etkiler: $2 \cdot 5 = 10$.
2. Geriye `3 10 + 1 +` kalır. `+` ile $3 + 10 = 13$ olur.
3. Geriye `13 1 +` kalır. Sonuç 14'tür.

Postfix ifade verilseydi derleyici tek yönden işleyerek doğru sonucu üretebilirdi. Infix'te ise işlem öncelikleri yüzünden sıralamayı yapamaz.

12.1 Operatör Öncelikleri

Stack ile dönüşüm yaparken operatör önceliklerinin bilinmesi gerekir: Çarpma mı önce, toplama mı? Kaydırma mı, üs alma mı? Parantez mi, mod alma mı, artırma mı? Hoca slaytta C için operatör öncelik tablosunu gösterdi. En yüksek öncelik en üstte, en düşük öncelik en alttadır. Yukarıdan aşağıya:

1. Parantez, dizi indisi, struct/union üyesi için `.` ve `->` (en yüksek öncelik).
2. Artırma ve azaltma.
3. Mantıksal değil, tümleyen, tekli eksi ve artı gibi tekli operatörler.
4. Tip dönüşümü (type casting).
5. Çarpma, bölme, mod (`* / %`).
6. Toplama, çıkarma.
7. Kaydırma (shift).
8. İlişkisel operatörler.
9. Eşitlik operatörleri.
10. En altta bit düzeyi operatörler, mantıksal operatörler, koşul operatörü ve atamalar.

Postfix'e çevirirken bu önceliklere dikkat etmek gerekir. Özellikle stack kullanılıyorsa.

13. Infix'ten Postfix'e: Parantezleme Yöntemi

Hoca önce "uzun, meşakkatli ve karmaşık yolu" gösterdi. İfade şuydu:

$$A/B - C + D \cdot E - A \cdot C$$

Hoca "bunu kim niye böyle yazar bilmiyorum" diye takıldı. Burada önce A/B, $D \cdot E$ ve $A \cdot C$ yapılmalı, sonra çıkan sonuçlarla toplama ve çıkarmalar yapılmalıdır.

Yöntem ifadeyi önceliklere göre tamamen parantezlemektir. Hoca bunu renklerle adım adım yaptı:

1. Bölme ve iki çarpma kendi parantezlerine alınır: $(A/B) - C + (D \cdot E) - (A \cdot C)$.
2. (Mavi) Soldan başlanır. $(A/B) - C$ bir parantez olur: $((A/B) - C)$.
3. (Yeşil) Buna $+(D \cdot E)$ eklenir: $(((A/B) - C) + (D \cdot E))$.
4. (Mor) Son çıkarma da dahil edilir: $((((A/B) - C) + (D \cdot E)) - (A \cdot C))$.

Sonra her operatör kendi parantezinin kapama (sağ) parantezinin yerine taşınır. Bölü, eksi, çarpı, artı, çarpı ve sondaki eksi kendi kapama parantezlerine gider. Son olarak bütün açma parantezleri silinir. Sonuç:

```
A B / C - D E * + A C * -
```

Operandların sırası infix ve postfix'te aynı kalır.

Hoca bu yöntemin uzun olduğunu söyledi. İşlem önceliklerine göre her şeyi iç içe parantezlere almak, sonra her operatörü sağ kapama paranteziyle yer değiştirmek ve açma parantezlerini silmek gerekir. Yapılabilir, ama özellikle parantezleme kısmı oldukça emek ister.

14. Infix'ten Postfix'e: Stack Yöntemi

Bunu daha kolay ve gerçekten bir algoritma biçiminde yapmak için bir veri yapısı gerekir: stack.

Bütün 17 öncelik düzeyi için yapılabilir. Basitlik için derste yalnız şu operatörler kullanıldı: açma ve kapama parantezi, `+`, `-`, `*`, `/` ve `%`. Terimler tek basamaklı tamsayılar ya da tek harfli değişkenlerdir. Böylece girdi karakter karakter işlenir. Hoca gerçekte girdilerin struct gibi daha karmaşık olabileceğini, ama anlatım için bu kabulü yaptığını söyledi.

Stack, infix'teki operatörleri işlem önceliklerine göre postfix'e taşımak için kullanılır. Kurallar:

- **Operand** (sayı ya da harf) okunduğu gibi çıktı katarına eklenir.
- **Açma parantezi** doğrudan push edilir.
- **Operatör** gelince: stack'in tepesindeki operatörün (peek edilen elemanın) önceliği, gelen operatörün önceliğinden büyük ya da eşit olduğu sürece tepedeki pop edilir ve çıktıya yazılır. Pop edilen atılmaz, çıktıya eklenir. Sonra gelen operatör push edilir.
- **Kapama parantezi** gelince açma parantezi görülene kadar stack'ten pop edilip çıktıya yazılır. Parantezler çıktıya yazılmaz.
- **Girdi bitince** (null karakter ya da string sonu) stack'te kalan bütün operatörler pop edilip çıkış sırasıyla çıktının sonuna eklenir.

  Ek bilgi (derste anlatılmadı): Yaygın gerçeklemelerde açma parantezi stack'e girdikten sonra en düşük öncelikli gibi davranır. Yani "tepedeki büyük ya da eşitse pop et" döngüsü bir açma parantezine gelince durur. Açma parantezi yalnız kapama parantezi geldiğinde stack'ten çıkar. Hocanın örneklerinde de parantezin içindeki operatörler bu şekilde parantezin üstüne push edildi.

14.1 Örnek 1: `A * (B + C) / D`

Hoca bu örneği slayttaki animasyonla gösterdi. Normalde çarpma ve bölme önceliklidir, ama toplama paranteze alınmıştır. Önce $B + C$ yapılmalı, sonuç A ile çarpılmalı ve çıkan sonuç D'ye bölünmelidir.

Okunan	Yapılan	Stack (alt → üst)	Çıktı
A	Operand, çıktıya		A
*	Stack boş, push	*	A
(	Push	* (	A
B	Operand, çıktıya	* (	A B
+	(üstüne push	* (+	A B
C	Operand, çıktıya	* (+	A B C
)	(görülene kadar pop: +	*	A B C +
/	* ile eşit öncelikli, * pop; / push	/	A B C + *
D	Operand, çıktıya	/	A B C + * D
son	Kalanlar pop		A B C + * D /

Kapama parantezinde +'nın pop edilmesi, B ve C operandlarını + ile öncelikli olarak işleme sokmak demektir. Sonuç `A B C + * D /` olur.

14.2 Örnek 2: `3 * (5 + 4 / 1) - 2`

Hoca ikinci örneği kendisi oluşturdu. İnsan olarak hesap: $4/1 = 4$, $5 + 4 = 9$, $3 \cdot 9 = 27$, $27 - 2 = 25$. Postfix ifade de bu 25 sonucunu verecek şekilde oluşmalıdır.

Okunan	Yapılan	Stack (alt → üst)	Çıktı
3	Operand, çıktıya		3
*	Stack boş, push	*	3
(	Push	* (	3
5	Operand, çıktıya	* (	3 5
+	(üstüne push	* (+	3 5
4	Operand, çıktıya	* (+	3 5 4
/	+'dan öncelikli, push	* (+ /	3 5 4
1	Operand, çıktıya	* (+ /	3 5 4 1
)	('ye kadar pop: /, +	*	3 5 4 1 / +
-	* daha öncelikli, pop; - push	-	3 5 4 1 / + *
2	Operand, çıktıya	-	3 5 4 1 / + * 2
son	Kalan - pop		3 5 4 1 / + * 2 -

Sonuç `3 5 4 1 / + * 2 -` olur. Bu postfix ifade hesaplandığında 25 çıkar.

> **Ders sonrası not:** Hoca + geldiğinde "artı zaten şeyden daha az öncelikli, normalde bunu pop etmem gerekirdi" dedi, sonra kapama paranteze kadar devam edileceği için +'yı push ettiğini söyledi. Doğru açıklama şudur: Tepede açma parantezi olduğu için hiçbir şey pop edilmez, + doğrudan push edilir. Ayrıca hoca kapama parantezinde operatörlerin "girdikleri sırayla" çıkarıldığını söyledi. Aslında stack'ten giriş sırasının tersiyle çıkarlar: önce /, sonra +. Hocanın tahtaya yazdığı çıktı da bu sıradadır.

15. Postfix İfadeyi Hesaplama

Derleyici tek yönden, soldan sağa işlem yapabiliyorsa verilen bir postfix ifadeyi nasıl işler? Yine bir stack kullanılır ve ifade tek geçişte taranır.

Dönüşümde sayılar doğrudan çıktıya gidiyor, operatörler stack'e giriyordu. Hesaplamada tam tersi yapılır:

- **Operand** görülünce stack'e push edilir.
- **Operatör** görülünce stack'ten son iki operand pop edilir, operatör uygulanır ve sonuç tekrar push edilir.
- **String sonunda** stack'te kalan değer pop edilir. Bu değer sonuçtur.

Ek bilgi (derste anlatılmadı): Operatör geldiğinde ilk pop edilen değer sağ operand, ikinci pop edilen değer sol operanddır. Toplama ve çarpmada fark etmez, ama bölme ve çıkarmada önemlidir. Örneğin `6 2 /` için önce 2, sonra 6 çekilir ve $6/2$ hesaplanır.

15.1 Örnek: `6 2 / 3 - 4 2 * +`

Okunan	Yapılan	Stack (alt → üst)	Alt ifade karşılığı
6	Push	6	
2	Push	6 2	
/	2 pop: $6/2 = 3$, push	3	$6/2$
3	Push	3 3	
-	2 pop: $3 - 3 = 0$, push	0	$6/2 - 3$
4	Push	0 4	
2	Push	0 4 2	
*	2 pop: $4 \cdot 2 = 8$, push	0 8	$4 \cdot 2$
+	2 pop: $0 + 8 = 8$, push	8	$6/2 - 3 + 4 \cdot 2$
son	Pop		Sonuç: 8

Hoca operatör geldiğinde iki eleman pop edildiği için stack pointer'ın iki azaldığını, sonuç push edilince bir arttığını gösterdi.

Stack'te sayısal sonuçlar yerine alt ifadelerin string'leri de tutulabilir. Hoca kırmızıyla değerleri, yanında alt ifadeleri yazdı. Stack'te ya `6/2-3` string'i ya da 0 değeri tutulur. Üstteki gözde ya `4*2` string'i ya da 8 tutulur. `+` gelince ikisi `+` ile birleşir. Böylece postfix ifade tekrar infix'e çevrilmiş olur: $6/2 - 3 + 4 \cdot 2$. Yani bu postfix ifade, $6/2 - 3 + 4 \cdot 2$ infix ifadesinin postfix karşılığıdır. Sonuç $3 - 3 + 8 = 8$'dir.

16. Büyük Örnek ve Hocanın Hesap Hatası

Hoca "iyice karmaşık" bir örnek daha yaptı:

$$4 \wedge 2 \cdot 3 - 3 + 8/4/(1 + 1)$$

Burada ^ üs alma olarak kullanıldı.

16.1 Elle Hesap

Hoca önce elle hesapladı: $4^2 = 16$, $16 \cdot 3 = 48$. Sonra $1 + 1 = 2$, "$4/2 = 2$", "$8/2 = 4$" dedi ve $48 - 3 + 4 = 49$ buldu. Ardından postfix'e dönüştürüp 49'u geri elde edip edemeyeceklerini görmek istedi.

16.2 Postfix'e Dönüşüm

Okunan	Yapılan	Stack (alt → üst)	Çıktı
4	Çıktıya		4
^	Stack boş, push	^	4
2	Çıktıya	^	4 2
*	^ daha öncelikli, pop; * push	*	4 2 ^
3	Çıktıya	*	4 2 ^ 3
-	* daha öncelikli, pop; - push	-	4 2 ^ 3 *
3	Çıktıya	-	4 2 ^ 3 * 3
+	- ile eşit, pop; + push	+	4 2 ^ 3 * 3 -
8	Çıktıya	+	4 2 ^ 3 * 3 - 8
/	+'dan öncelikli, push	+ /	4 2 ^ 3 * 3 - 8
4	Çıktıya	+ /	4 2 ^ 3 * 3 - 8 4

Okunan	Yapılan	Stack (alt → üst)	Çıktı
/	/ ile eşit, pop; / push	+ /	4 2 ^ 3 * 3 - 8 4 /
(	Push	+ / (	4 2 ^ 3 * 3 - 8 4 /
1	Çıktıya	+ / (	… 8 4 / 1
+	(üstüne push	+ / (+	… 8 4 / 1
1	Çıktıya	+ / (+	… 8 4 / 1 1
)	(’ye kadar pop: +	+ /	… 8 4 / 1 1 +
son	Kalanlar pop: /, +		… 8 4 / 1 1 + / +

Sonuç:

```
4 2 ^ 3 * 3 - 8 4 / 1 1 + / +
```

16.3 Postfix’i Hesaplama

Okunan	Yapılan	Stack (alt → üst)
4, 2	Push	4 2
^	$4^2 = 16$	16
3	Push	16 3
*	$16 \cdot 3 = 48$	48
3	Push	48 3
-	$48 - 3 = 45$	45
8, 4	Push	45 8 4
/	$8/4 = 2$	45 2
1, 1	Push	45 2 1 1
+	$1 + 1 = 2$	45 2 2
/	$2/2 = 1$	45 1
+	$45 + 1 = 46$	46

String sonunda stack’teki değer 46’dır.

16.4 Hata Nerede?

Hoca “başta kaç demiştik?” diye sordu. Bir öğrenci 49 dediklerini, elle hesapta işlem önceliğine uyulmadığını söyledi: $8/4$’ten 2 gelmeli, sonra $2/2$’den 1 gelmeliydi. Elle hesapta ise önce $4/2$ yapılıp $8/2 = 4$ bulunmuştu.

Aynı öncelikli / operatörleri soldan sağa uygulanır:

$$8/4/2 = (8/4)/2 = 2/2 = 1$$

Doğru sonuç $48-3+1 = 46$’dır. Hoca öğrencilere hatayı neden o anda söylemediklerini sordu. Öğrenci anlatımı bölmek istemediğini söyledi. Hoca bunu “insan şaşırır, postfix dönüşümü şaşırmaz” diye bağladı ve 46’nın doğru sonuç olduğunu söyledi.

> **Hocanın vurgusu:** Hoca öğrencilerin böyle bir hatayı ders sırasında hemen söylemesini istedi.
>
> **Ders sonrası not:** Örnekte ^ üs alma olarak kullanıldı. C’de ^ üs alma değil, bit düzeyinde XOR operatörüdür. Üs almak için pow fonksiyonu kullanılır. Ayrıca üs alma genelde sağdan birleşir: $2 \wedge 3 \wedge 2 = 2^{(3^2)}$. Bu yüzden ardışık ^ operatörlerinde “tepedeki büyük ya da eşitse pop et” kuralı uygulanmaz. Bu örnekte tek bir ^ olduğu için sonuç değişmez.

17. Stack İşlemlerinin Karmaşıklığı

- **push ve pop:** Linked list ise yeni düğüm oluşturup pointer'ları güncellemek ya da değeri alıp düğümü `free` etmektir. Dizi ise stack pointer'ı bir artırıp değeri yazmak ya da değeri alıp pointer'ı bir azaltmaktır. Eleman sayısından bağımsız, sabit sayıda işlem yapılır. Karmaşıklık $O(1)$'dir.
- **peek:** Yalnız `return top` yapılır. $O(1)$.
- **isEmpty / isFull:** Pointer -1 mi ya da en üst indiste mi diye bakılır. $O(1)$.

Stack'in atomik işlemleri sabit maliyetli olduğu için algoritmalara ek karmaşıklık yükü getirmez. Hocanın ifadesiyle "çarpan etkisi bir"dir. Örneğin n tane push yapılırsa toplam maliyet $O(n)$, $\log n$ tane pop yapılırsa $O(\log n)$ olur.

> **Hocanın vurgusu:** Stack işlemlerinin hepsi $O(1)$'dir. Bu yüzden LIFO ihtiyacı duyulan her yerde ek yük getirmeden kullanılabilir. Gündelik hayattaki çözümlerde bile stack'ten sıklıkla yararlanılır.

18. Kuyruk (Queue) Veri Yapısı

Aradan sonra hoca queue veri yapısına geçti. Queue stack'ten biraz farklıdır. LIFO yerine **FIFO** (first in, first out, ilk giren ilk çıkar) düzeni vardır. Elemanlar bir uçtan eklenir, diğer uçtan çıkarılır.

Hocanın benzetmesi otobüs ya da uçak kuyruğuydu. Kuyruğa ilk giren ilk çıkar, ikinci giren ikinci, üçüncü giren üçüncü çıkar.

Push ve pop yerine şu işlemler tanımlıdır:

- **enqueue:** Kuyruğa eleman ekleme.
- **dequeue:** Kuyruktan eleman çekip kullanma.

Queue, FIFO gereken bütün organizasyonlarda kullanılır:

- İşletim sistemi round robin süreç zamanlaması kullanıyorsa doğrudan bir FIFO kuyruğu kullanır.
- G/Ç (I/O) işleme yaparken FIFO kuyruğu kullanılır.
- Bir ağda router üzerinde paket iletiminde paketlere ilk gelen ilk servis alır biçiminde hizmet verilir.

Hoca kuyruğun fiziksel gerçeklemelerde de pek çok yerde kullanıldığını söyledi.

Kuyruk da stack gibi iki yolla oluşturulur:

- **Dizi ile:** Sabit boyutlu kuyruk. En kolay ve temiz yol budur, ama maliyetlidir. Dairesel dizi (circular array) ile kurulursa enqueue ve dequeue maliyeti sabite indirilir. Yoksa $O(n)$ mertebesine çıkma tehlikesi vardır.
- **Linked list ile:** Boyut dinamik olarak artar. Kuyruk teorik olarak hiç dolmaz, yalnız bellek alanıyla sınırlıdır.

19. Dizi ile Kuyruk ve Dairesel Dizi

19.1 Basit Çözüm ve Maliyeti

10 gözlük bir dizi açılsın. Başta kuyruk boştur.

1. enqueue 3: 3 birinci göze yerleşir.
2. enqueue 5: ikinci göze. enqueue 8: üçüncü göze. enqueue 1: dördüncü göze.
3. Yeni elemanlar geldikçe kuyrukta arkaya doğru ilerler.
4. dequeue: 3 kuyruktan alınır ve kullanılır. Birinci göz boş kalır.

Şimdi elemanları öne kaydırmak gerekir. Bu $n-1$ kaydırma işlemi demektir, yani $O(n)$ maliyet. Bir dequeue daha yapılırsa 5 gider, 8 ve 1 yine öne kayar. Yeni gelen 7 sona eklenir. Enqueue kolaydır: bir artır ve ekle, $O(1)$. Ama dequeue $O(n)$ maliyetlidir.

19.2 Dairesel Dizi (Circular Queue)

Dequeue maliyetini azaltmak için dizi dairesel hale getirilir. 10 göz ve iki uç vardır:

- **front:** Dequeue edilecek elemanın indisi.
- **rear:** En son enqueue edilen elemanın durduğu yer.
- **size:** Kuyruktaki eleman sayısı.

Örneğin size 4 ise ve front 0. indisi gösteriyorsa elemanlar 0, 1, 2, 3. indislerdedir ve rear 3'ü gösterir. `MAXSIZE` 10'dur. C olduğu için indisler 0'dan 9'a kadardır.

Derste başlangıç değerleri şöyle verildi: front -1, rear -1, size -1. Hoca bunu "başı da sonu da başlangıç noktasında duruyor" diye açıkladı.

- **isEmpty:** size sıfırdan küçük mü? Öyleyse kuyrukta eleman yoktur, true döner.
- **isFull:** size `MAXSIZE`'a eşit mi? Eşitse true, değilse false döner.

Kaydırmasız enqueue örneği: Front 0'ı gösteriyor, hiç dequeue yapılmamış. 4, 5, 6 ve 7 enqueue edilmiş, rear 3'tedir. enqueue 10 isteği gelince kuyruk dolu değilse rear bir artırılır ve 10 yazılır. Kuyrukta 4, 5, 6, 7, 10 vardır. size 5 olur. Front hâlâ 0, rear 4'tür.

Başa dönme örneği: Birçok enqueue ve dequeue işleminden sonra front 6, rear 9 olsun. size 4'tür. 10 gözden 4'ü dolu, 6'sı boştur. enqueue 10 gelince rear bir artırılır. Rear 10 olur, ama dizi dairesel olduğu için mod alındığında bu 0 demektir. Eleman 0. indise yazılır ve rear 0'ı gösterir.

Ardından bir dequeue yapılır: front'taki eleman alınır, size bir azaltılır, front bir ilerletilir. Rear hâlâ 0'dır. Yeni bir eleman (7) enqueue edilince size bir artırılır, rear 1'e çıkar ve 7 birinci indise yazılır. Front değişmez.

> **Ders sonrası not:** Hoca bu adımda "front 6 olarak kalmaya devam edecek" dedi. Ama hemen önce bir dequeue yapılmış ve front bir ilerletilmişti. Doğrusu, enqueue front'u değiştirmez ve front 7'de kalır. size de bu adımlarda $4 \to 5 \to 4 \to 5$ olarak değişir.

Front ve rear böylece döne döne birbirini takip eder. Enqueue rear'dan, dequeue front'tan yapılır. Front ya da rear dizinin sonuna (`MAXSIZE`'a) geldiğinde sıfırdan devam eder. Enqueue ve dequeue'da n tane kaydırma gerekmez. Yalnız size güncellenir, front ya da rear güncellenir ve oradaki değer eklenir ya da kullanılır. Bu üç sabit işlemdir. Bu yüzden enqueue, dequeue ve diğer bütün işlemler $O(1)$'dir.

19.3 enqueue ve dequeue Kodu

Slayttaki enqueue şöyle çalışır. Kuyruk dolu değilse:

- size sıfırdan küçükse hiç eleman yoktur. Değer `queue[0]`'a yazılır, front ve rear 0 yapılır, size 1 olur.
- rear `MAXSIZE - 1` ise rear dizinin sonundadır. Değer `queue[0]`'a yazılır, rear 0 yapılır, size bir artırılır.
- Diğer durumda değer rear'ın bir sonraki yerine yazılır, rear ve size bir artırılır.

Doluysa "Queue is full" denir.

```
void enqueue(int value) {
    if (size < MAXSIZE) {
        if (size < 0) {
            queue[0] = value;
            front = rear = 0;
            size = 1;
        } else if (rear == MAXSIZE - 1) {
            queue[0] = value;
            rear = 0;
            size++;
        } else {
            queue[rear + 1] = value;
            rear++;
```

```
            size++;
        }
    } else {
        printf("Queue is full\n");
    }
}
```

Dequeue'da geri döndürmek için bir `tmp` değişkeni gerekir. size sıfırdan küçükse kuyruk boştur ve hata döndürülür. Değilse `tmp = queue[front]` alınır, size bir azaltılır, front bir artırılır.

Hoca çıkarılan elemanları tahtada karalayıp nokta gibi gösterdi. Ama aslında o gözlerde değerler (örneğin 8, 10, 2, 3, 5, 6) durmaya devam eder. Kullanılmazlar, çünkü nereden çekileceği ve nereye ekleneceği front ve rear ile tutulur. Yeni bir eleman (örneğin 1) eklenirken rear ilerletilir ve eski değerin üzerine yazılır. Hoca isteyenin alternatif gerçeklemeler de yapabileceğini söyledi.

> **Ders sonrası not:** Derste size -1 ile başlatıldı ve boşluk `size < 0` ile kontrol edildi. Ama bütün elemanlar dequeue edildiğinde size 0 olur, -1 olmaz. Bu durumda isEmpty false döner ve boş kuyruktan dequeue yapılabilir (size -1'e düşer, eski bir değer döner). Hoca da bir yerde "size sıfırdan büyük eşitse tek bir eleman dahi olsa eleman var demektir" dedi; oysa size 0 iken eleman yoktur. Tutarlı çözüm size'ı 0 ile başlatıp boşluğu `size == 0` ile kontrol etmektir. Ayrıca dequeue'da front da başa dönmelidir: `front = (front + 1) % MAXSIZE`. Enqueue'daki dallar da tek satırda `rear = (rear + 1) % MAXSIZE` ile yazılabilir. (Kod `gcc -Wall` ile denendi: iki eleman ekleyip ikisini çıkarınca size 0 oldu ve isEmpty 0 döndü.)

20. Linked List ile Kuyruk

Linked list ile teorik olarak hiç dolmayan, sürekli büyüyebilen bir kuyruk kurulur. Yine fiziksel bellek sınırı vardır, ama teoride sınırsızdır.

20.1 Yapılar

Dizide de mutlaka üç bilgi tutuluyordu: nereden dequeue edileceği (front), nereye enqueue edileceği (rear) ve kaç eleman olduğu (size). Bu üç bilgi bir **Queue struct'ı** içinde tutulur:

- **count:** Kuyrukta kaç eleman var. Hoca buna çok gerek olmadığını, ama işi kolaylaştırdığını söyledi. "Kuyrukta kaç kişi var?" sorusuna doğrudan `return q->count` ile cevap verilir.
- **front:** Dequeue edilecek düğüm.
- **rear:** Enqueue edilecek yer, yani son düğüm.

Hocanın çiziminde kuyruğa önce A, sonra B, sonra C gelmişti. Front ilk gelen A'yı gösterir. A'nın next'i "benden sonra bu geldi" diye B'yi, B'nin next'i C'yi gösterir. C son gelendir ve en son çıkacaktır. Rear C'yi gösterir.

Her eleman için bir düğüm gerekir. Düğümde bir `data` ve `struct node *` tipinde bir `next` (ya da link) bulunur. Bu tip büyük N ile `Node` diye adlandırılmıştır. front ve rear `Node *` tipindedir.

- **init:** count 0, front NULL, rear NULL yapılır.
- **isEmpty:** `return q->rear == NULL`. Kuyruğun sonu (ya da başı) NULL'u gösteriyorsa hiç eleman yoktur ve true döner.

20.2 enqueue

Boş kuyruğa 4 eklenmek istensin:

1. Yeni bir `struct node` oluşturulur.
2. Verilen değer düğümün `data`'sına yazılır. Düğümün next'i NULL'dur.
3. Kuyruk boşsa front ve rear yeni düğümü gösterir. Tek eleman vardır, front ile rear aynı yerdedir.
4. Sayaç bir artırılır.

Kuyruk boş değilse, yani A eklenmiş ve front ile rear A'yı gösteriyorsa, yeni gelen B için yeni düğüm oluşturulur, next'i NULL yapılır ve değeri atanır. Sonra eski rear'ın next'i yeni düğüme bağlanır ve rear yeni düğümü gösterir. Front zaten A'yı göstermeye devam eder. C geldiğinde de aynı şekilde B'nin next'i C'ye bağlanır, rear C'ye güncellenir.

Kodda önce count'un `FULL`'dan küçük olup olmadığına bakılır. Belli bir sayıdan fazla eleman eklenmek istenmiyorsa "kuyruk doldu" denebilir.

```
void enqueue(Queue *q, int value) {
    if (q->count < FULL) {
        Node *tmp;
        tmp = malloc(sizeof(Node));
        tmp->data = value;
        tmp->next = NULL;
        if (!isempty(q)) {
            q->rear->next = tmp;
            q->rear = tmp;
        } else {
            q->front = q->rear = tmp;
        }
        q->count++;
    } else {
        printf("List is full\n");
    }
}
```

Özetle: düğüm oluştur, sona ekle, front, rear ve eski rear'ın next'ini güncelle, count'u artır. Eleman sayısına bağlı bir iş yapılmaz. Enqueue $O(1)$'dir.

20.3 dequeue

Dequeue edilecek yer front'tur. Adımlar:

1. Front'taki düğüm geçici bir `tmp` işaretçisiyle işaretlenir.
2. Front'un verisi döndürülmek üzere `n` değişkenine kopyalanır.
3. `q->front = q->front->next` ile front bir sonraki düğüme bağlanır.
4. Sayaç bir azaltılır.
5. `tmp` düğümü `free` edilir.

Hocanın çiziminde üç elemanlı kuyruktan bir eleman çıkarılınca iki elemanlı bir kuyruk kalır ve baştaki eleman kullanılmış olur.

```
int dequeue(Queue *q) {
    Node *tmp;
    int n = q->front->data;
    tmp = q->front;
    q->front = q->front->next;
    q->count--;
    free(tmp);
    return n;
}
```

> **Ders sonrası not:** Son düğüm çıkarılınca front NULL olur, ama rear hâlâ silinmiş düğümü gösterir. isEmpty rear'a baktığı için kuyruğu boş saymaz. Bir sonraki enqueue da silinmiş düğümün next'ine yazar. Çıkarmadan sonra `q->front == NULL` ise `q->rear = NULL` da yapılmalıdır. (Kod `gcc -Wall` ile denendi: üç eleman çıkarıldıktan sonra front NULL olduğu hâlde isEmpty 0 döndü.)

21. Kuyruk İşlemlerinin Karmaşıklığı

Dizi ile yapıldığında (normal dizi değil, front ve rear'a sahip dairesel dizi) ya da linked list ile yapıldığında, eleman sayısından bağımsız olarak yalnız birkaç işlem yapılır: iki pointer güncellemesi, bir sayaç değişikliği ya da bir düğüm yaratma/silme. Bu yüzden:

İşlem	Stack (dizi / linked list)	Queue (dairesel dizi / linked list)
Ekleme (`push` / `enqueue`)	$O(1)$	$O(1)$
Çıkarma (`pop` / `dequeue`)	$O(1)$	$O(1)$
`peek`	$O(1)$	$O(1)$
`isEmpty` / `isFull`	$O(1)$	$O(1)$

Queue'da peek, front'taki elemanın verisine bakmaktır: `return front->data`. Tek satırdır. isEmpty ve isFull da count'a ya da front ve rear'ın NULL olup olmadığına bakarak tek adımda yapılır.

Basit dizi ile kuyrukta ise dequeue, kaydırma yüzünden $O(n)$'dir.

> **Hocanın vurgusu:** Stack gibi queue'nun da bütün işlemleri $O(1)$'dir. Algoritma tasarımlarına ek karmaşıklık yükü bindirmez. Bu yüzden kullanmayı sevdiğimiz, tercih ettiğimiz bir veri yapısıdır.

Örnek Sorular

Soru 1 (derste sözlü, 00:13:46): Kapasitesi 6 olan bir stack'e 1'den 6'ya kadar sayılar push edilmiştir. 7. eleman push edilmeye çalışılırsa ne olur?

> **Çözüm:** Stack doludur. 7'yi push etmeye çalışmak **stack overflow** durumudur. isFull kontrolü yapılırsa bu engellenir.

Soru 2 (derste sözlü, 00:16:14): Aynı stack'ten art arda pop yapılırsa elemanlar hangi sırayla çıkar? Stack boşaldıktan sonra bir pop daha yapılırsa ne olur?

> **Çözüm:** Elemanlar 6, 5, 4, 3, 2, 1 sırasıyla çıkar. Boş stack'ten pop yapmak **stack underflow** durumudur. isEmpty true döndüğünde durulmalı ya da hata döndürülmelidir.

Soru 3 (derste sözlü, 00:36:17): "merhaba" string'i stack kullanarak nasıl tersine çevrilir?

> **Çözüm:** Null karakter görülene kadar karakterler push edilir. Sonra stack boşalana kadar pop edilir ve çıktıya yazılır: "abahrem". Sona null karakter eklenir. Karmaşıklık $O(n)$'dir.

Soru 4 (derste sözlü, 00:39:11): Bir string'in palindrom olup olmadığı stack ile nasıl kontrol edilir? String'in "yarısı" nedir? "kabak", "tabak" ve "küllük" için uygulayın.

> **Çözüm:** Uzunluk n ise ilk $\lfloor n/2 \rfloor$ karakter push edilir. n tekse ortadaki karakter atlanır. Kalan her karakter için bir pop yapılıp karşılaştırılır. Sonda string bitmiş ve stack boş olmalıdır.
>
> - kabak: k, a push edilir, b atlanır. a = a, k = k. Palindromdur.
> - tabak: t, a push edilir, b atlanır. a = a, k ≠ t. İlk uyuşmazlıkta durulur. Palindrom değildir.
> - küllük: k, ü, l push edilir. l = l, ü = ü, k = k. Palindromdur.

Soru 5 (derste sözlü, 01:11:05): `[ ( ( ) )` girdisinde bütün kapama parantezleri doğru açmayla eşleşiyor. Parantezler dengeli midir?

> **Çözüm:** Hayır. Girdi bittiğinde stack'te hâlâ `[` vardır. Girdi bitince stack'in boş olduğu mutlaka kontrol edilmelidir.

Soru 6 (derste sözlü, 01:44:20 ve 01:51:21): $4 \wedge 2 \cdot 3 - 3 + 8/4/(1+1)$ ifadesinin değeri nedir? Postfix karşılığı nedir?

> **Çözüm:** Postfix: `4 2 ^ 3 * 3 - 8 4 / 1 1 + / +`. Hesap: $16 \cdot 3 = 48$, $48 - 3 = 45$, $(8/4)/2 = 1$, $45 + 1 = 46$. Derste elle önce 49 bulundu, çünkü $8/4/2$ yanlışlıkla $8/(4/2)$ diye hesaplandı. Postfix hesabı 46'yı verdi ve doğru sonuç 46'dır. (Python ile doğrulandı.)

Dersin Toparlaması

Bu derste iki temel veri yapısı görüldü. Stack, tek uçtan erişim sağlayan LIFO (ya da FILO) bir yapıdır. İlk push edilen en altta, son push edilen en üstte kalır. Pop ederken elemanlar $n, n-1, \ldots, 1$ sırasıyla alınır. Stack dizi ile (top pointer ve isFull/isEmpty kontrolleriyle) ya da linked list ile (başa ekleme, baştan silme) kurulabilir. String ters çevirme, palindrom kontrolü, parantez dengesi kontrolü, infix ifadeyi postfix'e çevirme ve postfix ifadeyi tek geçişte hesaplama stack'in tipik kullanımlarıdır. Hocanın 49 bulduğu ama postfix hesabının 46 verdiği örnek, eşit öncelikli operatörlerin soldan sağa uygulanması gerektiğini gösterdi.

Queue ise elemanların bir uçtan girip öbür uçtan çıktığı FIFO bir yapıdır. Basit dizi ile dequeue $O(n)$ kaydırma ister. Dairesel dizi ya da linked list ile enqueue ve dequeue $O(1)$ olur. Hoca iki yapının da bütün işlemlerinin $O(1)$ olduğunu, bu yüzden algoritmalara ek yük getirmediğini vurgulayarak dersi kapattı. Stack'in Türkçe adıyla karışabilen heap veri yapısının iki hafta sonra görüleceği ders başında belirtilmişti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.